

Project Report

Project Title: Blogsphere

Name: Antara Arifa Mullick

Id: 210042132

Semester: 5th

Course: Server Programming Lab SWE 4538

Submitted To: Farzana Tabassum

GitHub Link:

[BlogSite GitHub Repository]

Project Overview

Blogsphere is a full-stack web application designed for users to create, manage, and engage with blog content. It offers a secure and user-friendly environment where users can register, log in, post content, comment, and interact with others. The platform is built to make content sharing easy while maintaining secure authentication and user interactions. It solves the problem of providing a central hub for content creation and interaction in a streamlined and secure manner, ensuring that only authorized users can perform certain actions like posting and commenting.

Objectives:

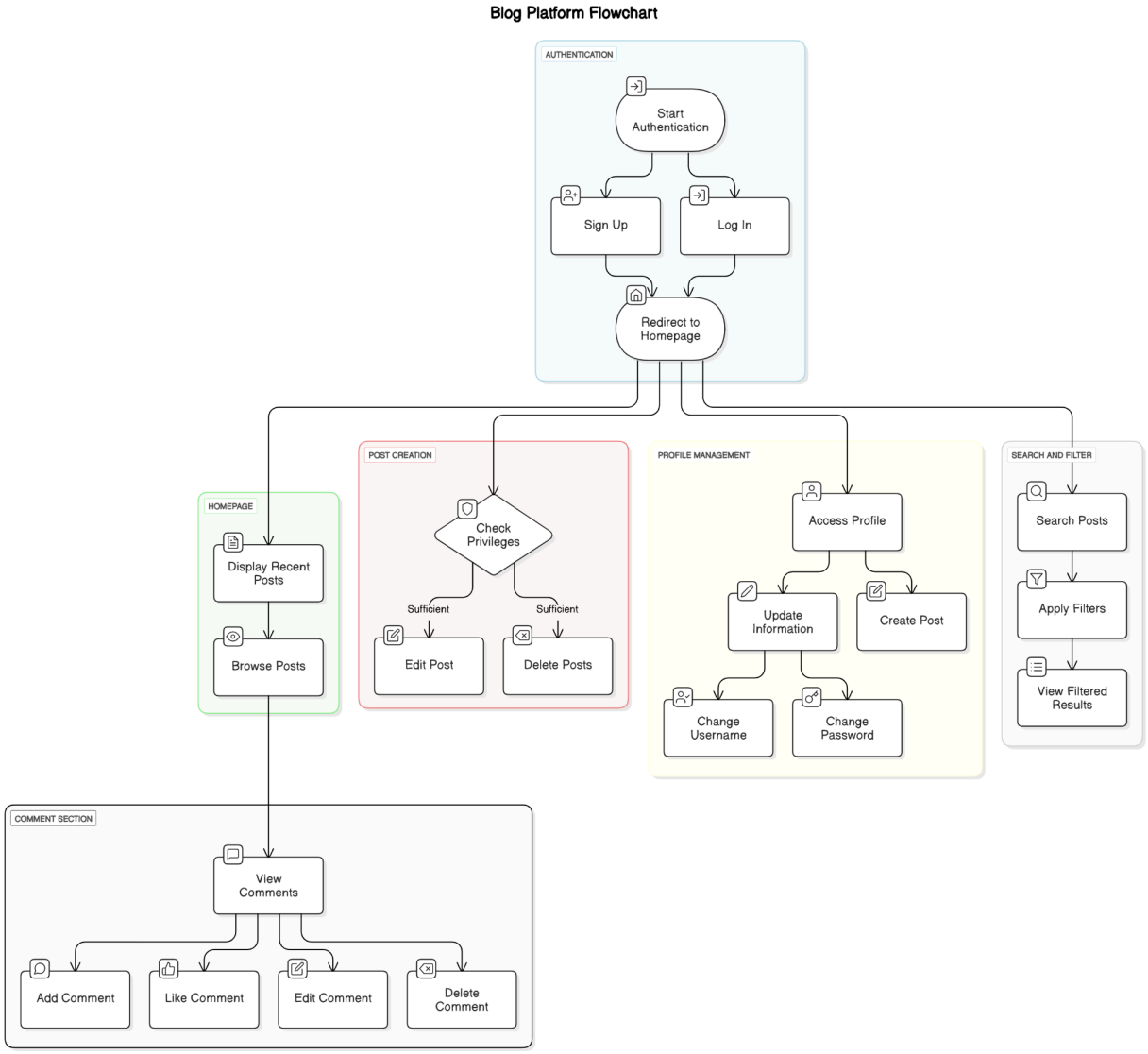
- To create a platform where users can post content, engage with other posts, and manage their own blogs.
- To implement secure authentication methods such as JWT, OAuth, and session-based authentication.
- To provide a seamless user experience with features like creating, editing, and deleting posts, commenting, liking comments, and user profile management.

Feature Description

1. **User Authentication**
    - Users can register and log in using email/password, JWT-based authentication, or Google OAuth.
    - Multi-level authentication ensures secure access to the application.
2. **Homepage**
    - Displays recent blog posts in a feed, allowing users to click and read more detailed content.
3. **Blog Post Management**
    - Users can create, edit, delete, and view blog posts. Only authorized users (admins or the post authors) can edit or delete posts.

4. **Commenting System**
    - Users can create, edit, and delete comments on blog posts.
    - Likes on comments allow for user interaction.
5. **Profile Management**
    - Users can update their personal profile information, including username, email, and password.
6. **Search and Filter**
    - Users can search for posts using keywords, filter by category, or search by the author.
7. **Admin Features**
    - Admins can manage posts and users, including deleting or editing posts from other users.

Navigation


Blog Platform Flowchart
AUTHENTICATION
Start Authentication
Sign Up
Log In
Redirect to Homepage
HOMEPAGE
Display Recent Posts
Browse Posts
POST CREATION
Check Privileges
Sufficient
Sufficient
Edit Post
Delete Posts
PROFILE MANAGEMENT
Access Profile
Update Information
Create Post
Change Username
Change Password
SEARCH AND FILTER
Search Posts
Apply Filters
View Filtered Results
COMMENT SECTION
View Comments
Add Comment
Like Comment
Edit Comment
Delete Comment

Tools and Technology

- **Frontend**:
    - **React.js**: A JavaScript library for building the user interface. React is used for its component-based architecture, which allows for a dynamic and responsive user interface.
    - **Tailwind CSS**: A utility-first CSS framework that helps in designing responsive and aesthetically pleasing layouts quickly.
- **Backend**:
    - **Node.js**: A JavaScript runtime that helps in building scalable and efficient backend services.
    - **Express.js**: A web application framework for Node.js, simplifying routing and middleware handling.
- **Database**:
    - **MongoDB**: A NoSQL database for storing user data, posts, comments, and other application data in a flexible format.
- **Authentication**:
    - **JWT (JSON Web Tokens)**: Used for securing the login system, ensuring that users' sessions are valid before allowing access to certain routes.
    - **OAuth**: Used for third-party login (e.g., Google login) to make the registration and login process more seamless.
    - **Session-Based Authentication**: Another form of user authentication that keeps track of logged-in users.

API Design

1. **Auth API**
    - **POST /signup**: Registers a new user. Takes `username`, `email`, and `password` as input.
    - **POST /signin**: Authenticates a user using `email` and `password`. Returns a JWT token if the credentials are valid.
    - **POST /google**: Handles Google OAuth login. Redirects to Google authentication and stores the user's credentials upon success.

2. **Post Management API**

    - **POST /create**: Allows authenticated users to create new blog posts.
    - **PUT /updatepost/:postId**: Allows the author of a post to update their post.
    - **DELETE /deletepost/:postId**: Allows the post author or an admin to delete a post.
    - **GET /getposts**: Retrieves all posts, optionally filtered by user, category, or search term.
3. **Comment Management API**

    - **POST /create**: Creates a comment for a blog post.
    - **PUT /likeComment/:commentId**: Allows users to like or unlike a comment.
    - **PUT /editComment/:commentId**: Allows users to edit their own comments.
    - **DELETE /deleteComment/:commentId**: Allows users to delete their own comments or an admin to delete any comment.
4. **Profile Management API**

    - **GET /profile**: Retrieves the current user's profile data.
    - **PUT /updateProfile**: Updates the user's profile information (email, username, password).

Use Case Scenarios

1. **User Logs In and Views Posts**

    - A user logs into their account using JWT or Google OAuth. After logging in, the user is redirected to the homepage where they can browse the latest blog posts.
2. **User Creates a Post**

    - A user with admin privileges creates a new post by filling out a form with a title, content, and category. The post is then stored in the database and made available for other users to view.
3. **User Comments on a Post**

- A user comments on a post and can edit or delete their comment. Other users can like the comment to show approval.

Challenges & Solutions

1. **Authentication Management**
    - **Challenge**: Implementing both session-based and token-based authentication could cause conflicts in managing sessions and protecting routes.
    - **Solution**: Clear separation of the authentication routes for JWT and session-based login, with middleware used to verify tokens for securing routes.
2. **Handling Comments Dynamically**
    - **Challenge**: Handling a dynamic commenting system where users can add, edit, and delete comments while ensuring accurate like/dislike counts.
    - **Solution**: Implemented efficient state management with React and dynamic API calls to update the UI in real-time.
3. **Database Optimization**
    - **Challenge**: Managing a large number of blog posts and comments could result in slow performance when retrieving data.
    - **Solution**: Introduced pagination and indexing in MongoDB to speed up queries and ensure smooth performance.

Future Improvements

1. **Real-Time Chat:**
    - Implement a real-time chat feature between users to facilitate discussions about various blogposts.
2. **Post Image Support**
    - Add functionality for users to upload images with their posts to make the content more visually engaging.

3. **Advanced Search Functionality**

    - Implement full-text search and tag-based filtering to improve the search experience, allowing users to find posts more effectively.

Conclusion

The Blogsphere project allowed me to create a secure and feature-rich platform where users can interact with blog posts and comments. By implementing various authentication mechanisms and building dynamic features like post and comment management, I learned how to develop a full-stack web application. The project significantly improved my understanding of both frontend and backend technologies, and I am excited to continue enhancing the platform with real-time features and other integrations.